Claude Agent Skills: A First Principles Deep Dive

Oct 26, 2025 • Han Lee | 41 min read (7455 words)

Claude's Agent `Skills` system represents a sophisticated prompt-based meta-tool architecture that extends LLM capabilities through specialized instruction injection. Unlike traditional function calling or code execution, `skills` operate through **prompt expansion** and **context modification** to modify how Claude processes subsequent requests without writing executable code.

This deep dive deconstructs Claude's Agent `Skills` system from first principles, documents the architecture where a tool named "`Skill`" acts as a meta-tool for injecting domain-specific prompts into the conversation context. We'll walk through the complete lifecycle using the `skill-creator` and `internal-comms` skill as case studies, examining everything from file parsing to API request structure to Claude's decision-making process.

Claude Agent Skills Overview

Claude uses `Skills` to improve how it performs specific tasks. `Skills` are defined as folders that include instructions, scripts, and resources that Claude can load when needed. Claude uses a **declarative, prompt-based system** for skill discovery and invocation. The AI model (Claude) makes the decision to invoke `skills` based on textual descriptions presented in its system prompt. **There is no algorithmic `skill` selection or AI-powered intent detection** at the code level. The decision-making happens entirely within Claude's reasoning process based on the skill descriptions provided.

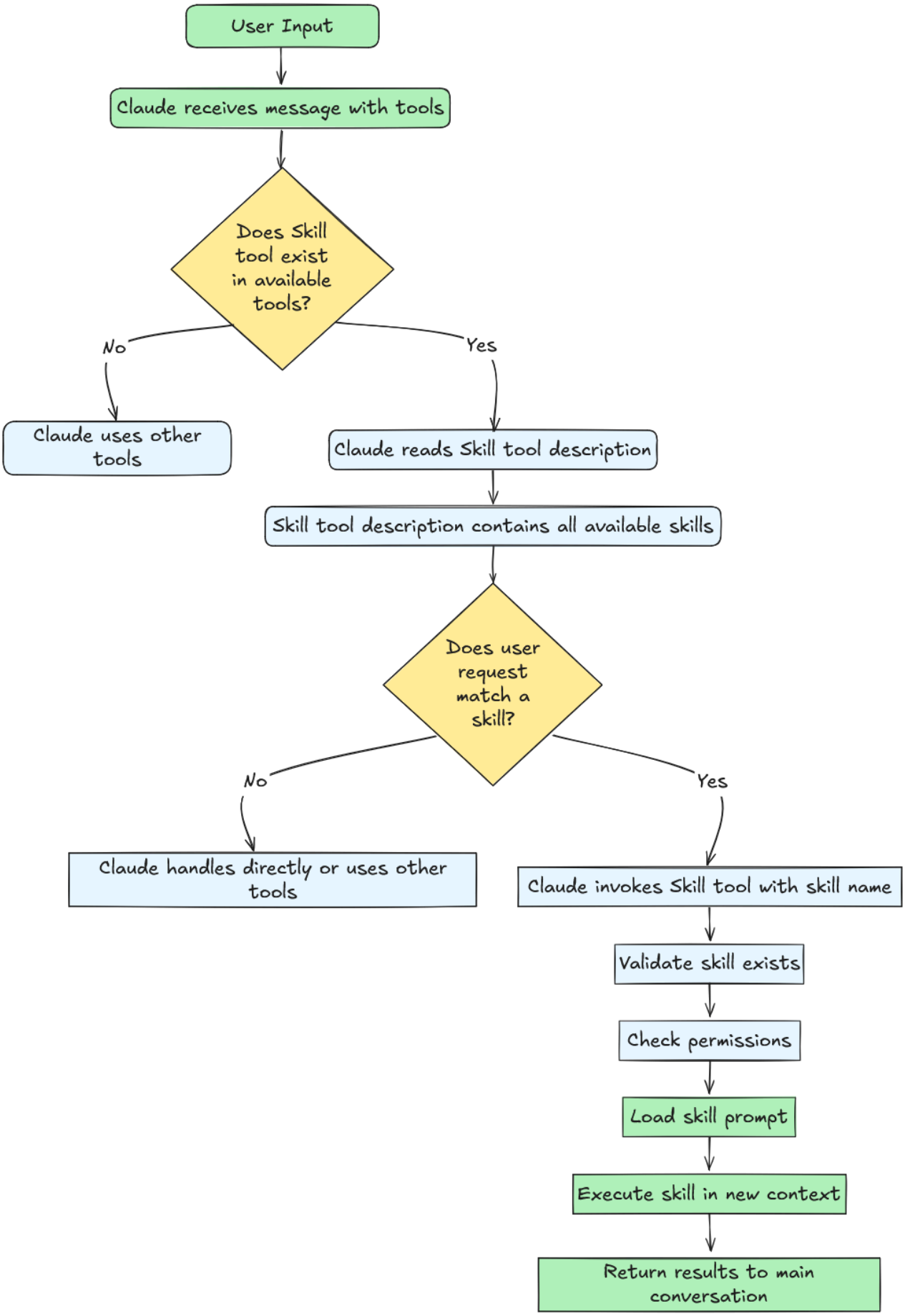
`Skills` are not executable code. They do **NOT** run Python or JavaScript, and there's no HTTP server or function calling happening behind the scenes. They are also not hardcoded into Claude's system prompt. `Skills` live in a separate part of the API request structure.

So what are they? `Skills` are specialized prompt templates that inject domain-specific instructions into the conversation context. When a skill is invoked, it modifies both the conversation context (by injecting instruction prompts) and the execution context (by changing tool permissions and potentially switching the model). Instead of executing actions directly, skills expand into detailed prompts that prepare Claude to solve a specific type of problem. Each skill appears as a dynamic addition to the tool schema that Claude sees.

When users send a request, Claude receives three things: user message, the available tools (Read, Write, Bash, etc.), and the `Skill` tool. The `Skill` tool's description contains a formatted list of every available skill with their `name`, `description`, and other fields combined. Claude reads this list and uses its native language understanding to match your intent against the skill descriptions. If you say "help me create a skill for logs," Claude sees the `internal-comms` skill's description ("When user wants to write internal communications using format that his company likes to use"), recognizes the match, and invokes the `Skill` tool with `command: "internal-comms"`.

> **Terminology Note**:
>
> - `Skill` **tool** (capital S) = The meta-tool that manages all skills. It appears in Claude's `tools` array alongside Read, Write, Bash, etc.
> - **skills** (lowercase s) = Individual skills like `pdf`, `skill-creator`, `internal-comms`. These are the specialized instruction templates that the `Skill` tool loads.

Here's a more visual representation on `skills` are used by Claude.



The skill selection mechanism has no algorithmic routing or intent classification at the code level. Claude Code doesn't use embeddings, classifiers, or pattern matching to decide which skill to invoke. Instead, the system formats all available skills into a

text description embedded in the `Skill` tool's prompt, and lets Claude's language model make the decision. This is pure LLM reasoning. No regex, no keyword matching, no ML-based intent detection. The decision happens inside Claude's forward pass through the transformer, not in the application code.

When Claude invokes a skill, the system follows a simple workflow: it loads a markdown file (`SKILL.md`), expands it into detailed instructions, injects those instructions as new user messages into the conversation context, modifies the execution context (allowed tools, model selection), and continues the conversation with this enriched environment. This is fundamentally different from traditional tools, which execute and return results. Skills *prepare Claude* to solve a problem, rather than solving it directly.

The following is a table to help better disambiguating the difference between Tools and Skills and their capabilities:

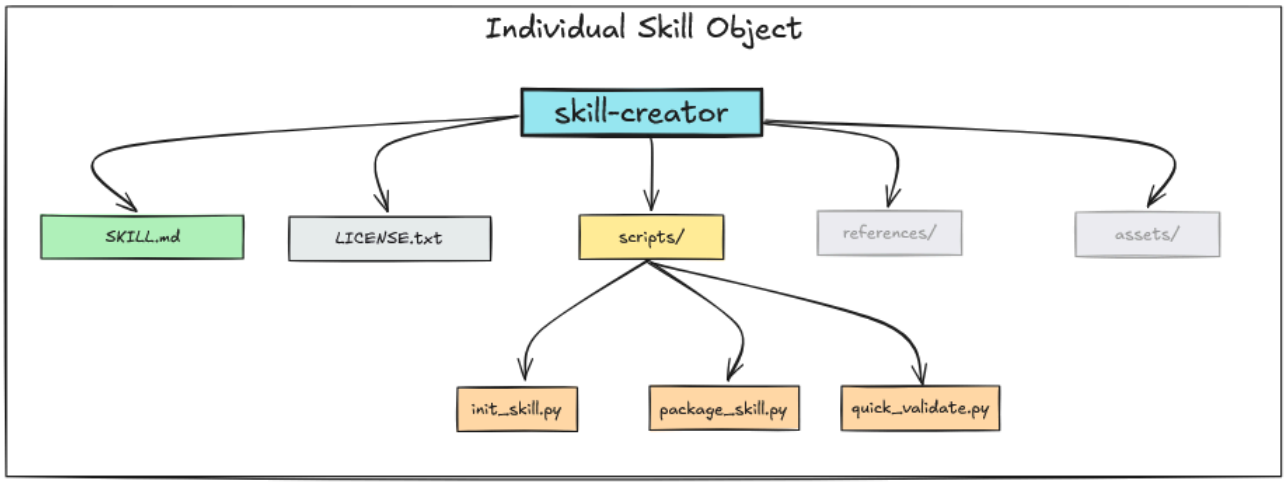
Aspect	Traditional Tools	Skills
Execution Model	Synchronous, direct	Prompt expansion
Purpose	Perform specific operations	Guide complex workflows
Return Value	Immediate results	Conversation context + execution context changes
Example	`Read` , `Write` , `Bash`	`internal-comms` , `skill-creator`
Concurrency	Generally safe	Not concurrency-safe
Type	Various	Always `"prompt"`

Building Agent Skills

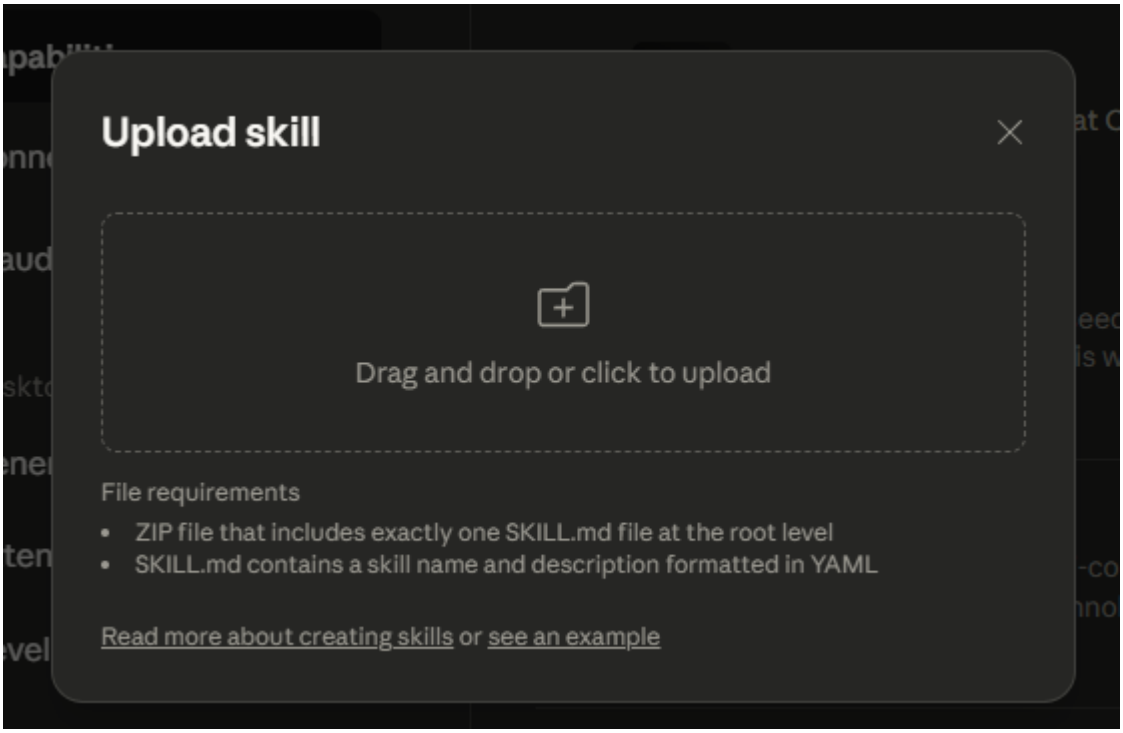
Now lets dive into how to build Skills by examining the `skill-creator` Skill from Anthropic's skill repository as a case study. As a reminder, agent `skills` are organized folders of instructions, scripts, and resources that agents can discover and load dynamically to perform better at specific tasks. `Skills` extend Claude's capabilities by packaging your expertise into composable resources for Claude, transforming general-purpose agents into specialized agents that fit your needs.

> **Key Insight**: Skill = Prompt Template + Conversation Context Injection + Execution Context Modification + Optional data files and Python Scripts

Every `Skill` is defined in a markdown file named `SKILL.md` (case-insensitive) with optional bundled files that's stored under `/scripts`, `/references`, and `/assets`. These bunlded files can be Python scripts, shell scripts, font definitions, templates, etc. Using `skill-creator` as an example, it contains `SILL.md`, `LICENSE.txt` for the license, and a few Python scripts under teh `/scripts` folder. `skill-creator` does not have any `/references` or `/assets`.



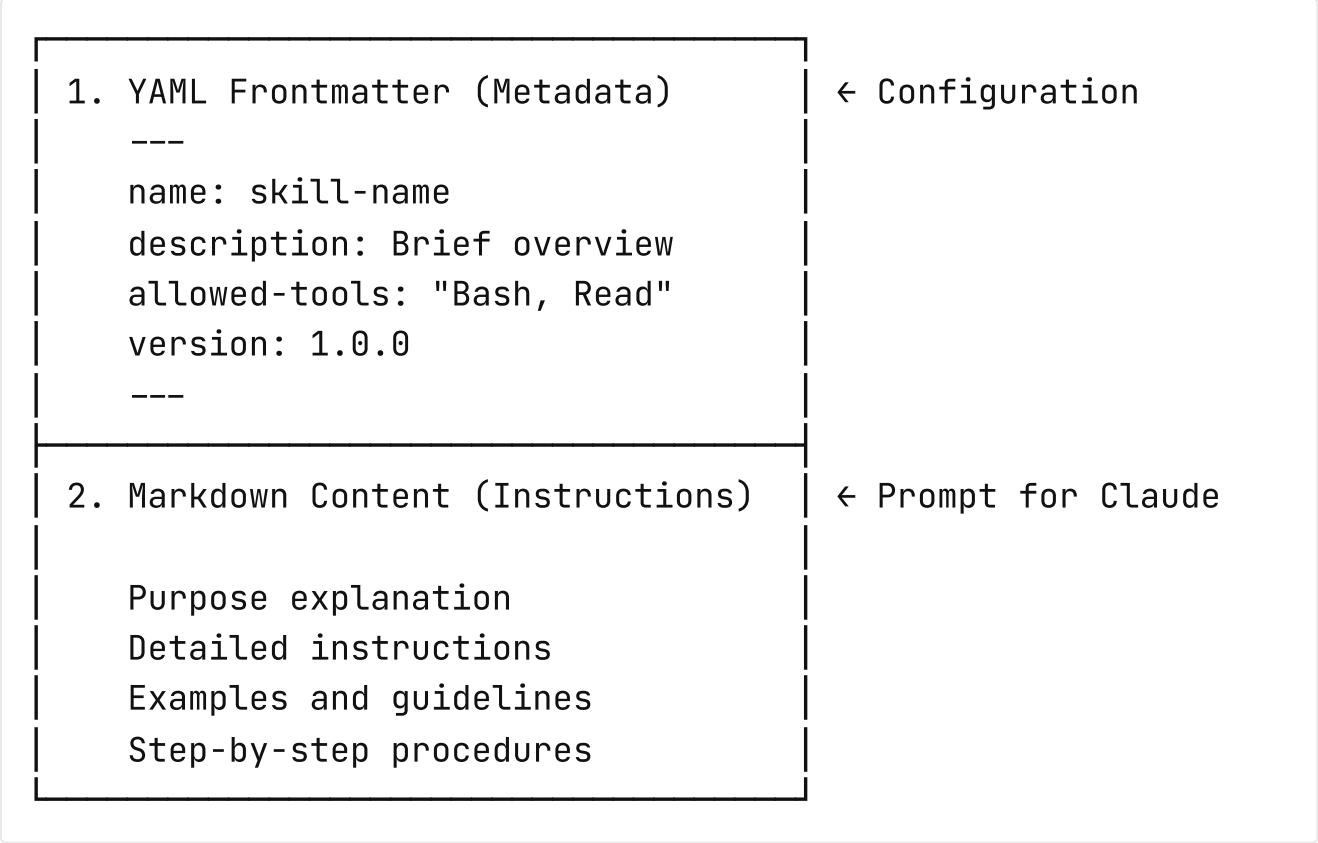
Skills are discovered and loaded from multiple sources. Claude Code scans user settings (`~/.config/claude/skills/`), project settings (`.claude/skills/`), plugin-provided skills, and built-in skills to build the available skills list. For Claude Desktop, we can upload a custom skill as follows.



> **NOTE:** The most important concept for building Skills is **Progressive Disclosure** - showing just enough information to help agents decide what to do next, then reveal more details as they need them. In the case of `agent skills`, it
>
> 1. Disclose Frontmatter: minimal (name, description, license)
> 2. If a `skill` is chosen, load SKILL.md: comprehensive but focused
> 3. And then load helper assets, references, and scripts as the `skill` is being executed

Writing SKILL.md

`SKILL.md` is the core of an skill's prompt. It is a markdown file that follows a two-part structure - frontmatter and content. The frontmatter configures HOW the skill runs (permissions, model, metadata), while the markdown content tells Claude WHAT to do. Frontmatter is the header of the markdown file written in YAML.



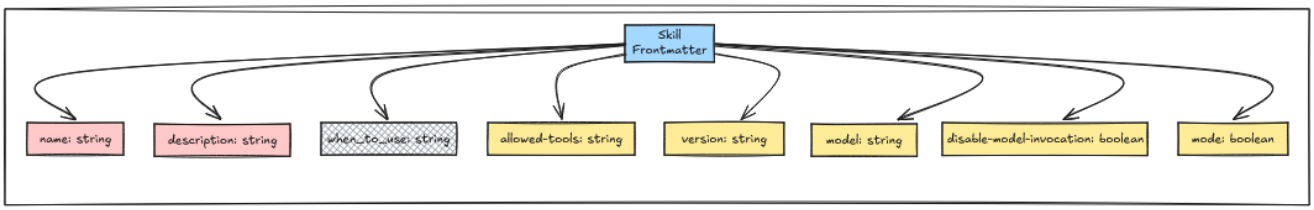
Frontmatter

The frontmatter contains metadata that controls how Claude discovers and uses the skill. As an example, here's the frontmatter from `skill-creator`:

```
---
name: skill-creator
description: Guide for creating effective skills. This skill sho
```

```
license: Complete terms in LICENSE.txt
---
```

Lets walk through the fields for the frontmatter one by one.



`name` (Required)

Self explanatory. Name of the `skill`. The `name` of a `skill` is used as a `command` in `Skill Tool`.

> The `name` of a `skill` is used as a `command` in `Skill Tool`.

`description` (Required)

The `description` field provides a brief summary of what the skill does. This is the primary signal Claude uses to determine when to invoke a skill. In the example above, the description explicitly states "This skill should be used when users want to create a new skill" — this type of clear, action-oriented language helps Claude match user intent to skill capabilities.

The system automatically appends source information to the description (e.g., `"(plugin:skills)"`), which helps distinguish between skills from different sources when multiple skills are loaded.

`when_to_use` (Undocumented—Likely Deprecated or Future Feature)

> ⚠️ **Important Note**: The `when_to_use` field appears extensively in the codebase but is **not documented in any official Anthropic documentation**. This field may be:
>
> - A deprecated feature being phased out
> - An internal/experimental feature not yet officially supported
> - A planned feature that hasn't been released
>
> **Recommendation**: Rely on a detailed `description` field instead. Avoid using `when_to_use` in production skills until it appears in official documentation.

Despite being undocumented, here's how `when_to_use` currently works in the codebase:

```
function formatSkill(skill) {
  let description = skill.whenToUse
    ? `${skill.description} - ${skill.whenToUse}`
    : skill.description;

  return `"${skill.name}": ${description}`;
}
```

When present, `when_to_use` gets appended to the description with a hyphen separator. For example:

```
"skill-creator": Create well-structured, reusable skills... - Wh
```

This combined string is what Claude sees in the Skill tool's prompt. However, since this behavior is undocumented, it could change or be removed in future releases. The safer approach is to include usage guidance directly in the `description` field, as shown in the `skill-creator` example above.

`license` (Optional)

Self explanatory.

`allowed-tools` (Optional)

The `allowed-tools` field defines which tools the skill can use without user approval, similar to Claude's allowed-tools.

This is a comma-separated string that gets parsed into an array of allowed tool names. You can use wildcards to scope permissions, e.g., `Bash(git:*)` allows only git subcommands, while `Bash(npm:*)` permits all npm operations. The skill-creator skill uses `"Read,Write,Bash,Glob,Grep,Edit"` to give it broad file and search capabilities. A common mistake is listing every available tool, which creates a security risk and defeats the security model.

> Only include what your skill actually needs—if you're just reading and writing files, `"Read,Write"` is sufficient.

```
# ✅ skill-creator allows multiple tools
allowed-tools: "Read,Write,Bash,Glob,Grep,Edit"

# ✅ Specific git commands only
allowed-tools: "Bash(git status:*),Bash(git diff:*),Bash(git log

# ✅ File operations only
allowed-tools: "Read,Write,Edit,Glob,Grep"

# ❌ Unnecessary surface area
allowed-tools: "Bash,Read,Write,Edit,Glob,Grep,WebSearch,Task,Ag

# ❌ Unnecessary surface area with all npm commands
allowed-tools: "Bash(npm:*),Read,Write"
```

`model` (Optional)

The `model` field defines which model the skill can use. It defaults to inheriting the current model in the user session. For complex tasks like code review, skills can request more capable models such as Claude Opus or other OSS Chinese models. IYKYK.

```
model: "claude-opus-4-20250514"  # Use specific model
model: "inherit"                 # Use session's current model (
```

`version`, `disable-model-invocation`, and `mode` (Optional)

Skills support three optional frontmatter fields for versioning and invocation control. The `version` field (e.g., version: "1.0.0") is a metadata field for tracking skill versions, parsed from the frontmatter but primarily used for documentation and skill management purposes.

The `disable-model-invocation` field (boolean) prevents Claude from automatically invoking the skill via the `Skill` tool. When set to true, the skill is excluded from the list shown to Claude and can only be invoked manually by users via `/skill-name`, making it ideal for dangerous operations, configuration commands, or interactive workflows that require explicit user control.

The `mode` field (boolean) categorizes a skill as a "mode command" that modifies Claude's behavior or context. When set to true, the skill appears in a special "Mode

Commands" section at the top of the skills list (separate from regular utility skills), making it prominent for skills like debug-mode, expert-mode, or review-mode that establish specific operational contexts or workflows.

SKILL.md Prompt Content

After the frontmatter comes the markdown content - the actual prompt that Claude receives when the `skill` is invoked. This is where you define the `skill`'s behavior, instructions, and workflows. The key to writing effective skill prompts is keeping them focused and using progressive disclosure: provide core instructions in SKILL.md, and reference external files for detailed content.

Here's a recommended content structure

```
---
# Frontmatter here
---

# [Brief Purpose Statement - 1-2 sentences]

## Overview
[What this skill does, when to use it, what it provides]

## Prerequisites
[Required tools, files, or context]

## Instructions

### Step 1: [First Action]
[Imperative instructions]
[Examples if needed]

### Step 2: [Next Action]
[Imperative instructions]

### Step 3: [Final Action]
[Imperative instructions]

## Output Format
[How to structure results]

## Error Handling
```

```
[What to do when things fail]

## Examples
[Concrete usage examples]

## Resources
[Reference scripts/, references/, assets/ if bundled]
```

As an example, `skill-creator` skill contains the following instructions that specifies each steps of the workflow required to create skills.

```
## Skill Creation Process

### Step 1: Understanding the Skill with Concrete Examples
### Step 2: Planning the Reusable Skill Contents
### Step 3: Initializing the Skill
### Step 4: Edit the Skill
### Step 5: Packaging a Skill
```

When Claude invokes this skill, it receives the entire prompt as new instructions with the base directory path prepended. The `{baseDir}` variable resolves to the skill's installation directory, allowing Claude to load reference files using the Read tool: `Read({baseDir}/scripts/init_skill.py)`. This pattern keeps the main prompt concise while making detailed documentation available on demand.

Best practices for prompt content:

- Keep under 5,000 words (~800 lines) to avoid overwhelming context
- Use imperative language ("Analyze code for…") not second person ("You should analyze…")
- Reference external files for detailed content rather than embedding everything
- Use `{baseDir}` for paths, never hardcode absolute paths like `/home/user/project/`

```
❌ Read /home/user/project/config.json
✅ Read {baseDir}/config.json
```

When the skill is invoked, Claude receives access only to the tools specified in `allowed-tools`, and the model may be overridden if specified in the frontmatter. The skill's base directory path is automatically provided, making bundled resources accessible.

Bundling Resources with Your Skill

`Skills` become powerful when you bundle supporting resources alongside SKILL.md. The standard structure uses three directories, each serving a specific purpose:

```
my-skill/
├── SKILL.md              # Core prompt and instructions
├── scripts/              # Executable Python/Bash scripts
├── references/           # Documentation loaded into context
└── assets/               # Templates and binary files
```

Why bundle resources? Keeping SKILL.md concise (under 5,000 words) prevents overwhelming Claude's context window. Bundled resources let you provide detailed documentation, automation scripts, and templates without bloating the main prompt. Claude loads them only when needed using progressive disclosure.

The `scripts/` Directory

The `scripts/` directory contains executable code that Claude runs via the Bash tool—automation scripts, data processors, validators, or code generators that perform deterministic operations.

As an example, `skill-creator`'s SKILL.md reference scripts like this:

```
When creating a new skill from scratch, always run the `init_ski

Usage:

```scripts/init_skill.py <skill-name> --path <output-directory>`

The script:
  - Creates the skill directory at the specified path
  - Generates a SKILL.md template with proper frontmatter and TO
  - Creates example resource directories: scripts/, references/,
  - Adds example files in each directory that can be customized
```

When Claude sees this instruction, it executes `python {baseDir}/scripts/init_skill.py`. The `{baseDir}` variable automatically resolves to the skill's installation path, making the skill portable across different environments.
```

Use scripts/ for complex multi-step operations, data transformations, API interactions, or any task requiring precise logic better expressed in code than natural language.

The `references/` Directory

The `references/` directory stores documentation that Claude reads into its context when referenced. This is text content—markdown files, JSON schemas, configuration templates, or any documentation Claude needs to complete the task.

As an example, `mcp-creator`'s SKILL.md reference references like this:

```
##### 1.4 Study Framework Documentation

**Load and read the following reference files:**

- **MCP Best Practices**: [📋 View Best Practices](./reference/r

**For Python implementations, also load:**
- **Python SDK Documentation**: Use WebFetch to load `https://ra
- [🐍 Python Implementation Guide](./reference/python_mcp_server

**For Node/TypeScript implementations, also load:**
- **TypeScript SDK Documentation**: Use WebFetch to load `https:
- [⚡ TypeScript Implementation Guide](./reference/node_mcp_serve
```

When Claude encounters these instructions, it uses the Read tool: `Read({baseDir}/references/mcp_best_practices.md)`. The content gets loaded into Claude's context, providing detailed information without cluttering SKILL.md.

Use references/ for detailed documentation, large pattern libraries, checklists, API schemas, or any text content that's too verbose for SKILL.md but necessary for the task.

The `assets/` Directory

The `assets/` directory contains templates and binary files that Claude references by path but doesn't load into context. Think of this as the skill's static resources - HTML templates, CSS files, images, configuration boilerplate, or fonts.

In SKILL.md:

```
Use the template at {baseDir}/assets/report-template.html as the
Reference the architecture diagram at {baseDir}/assets/diagram.p
```

Claude sees the file path but doesn't read the content. Instead, it might copy the template to a new location, fill in placeholders, or reference the path in generated output.

Use assets/ for HTML/CSS templates, images, binary files, configuration templates, or any file that Claude manipulates by path rather than reads into context.

The key distinction between `references/` and `assets/` are that

- **references/**: Text content loaded into Claude's context via Read tool
- **assets/**: Files referenced by path only, not loaded into context

This distinction matters for context management. A 10KB markdown file in `references/` consumes context tokens when loaded. A 10KB HTML template in `assets/` does not. Claude just knows the path exists.

> **Best practice:** Always use `{baseDir}` for paths, never hardcode absolute paths. This makes skills portable across user environments, project directories, and different installations.

Common Skill Patterns

As with everything engineering, understanding common patterns helps in design effective skills. Here are the most useful patterns for tool integration and workflow design.

Pattern 1: Script Automation

Use case: Complex operations requiring multiple commands or deterministic logic.

This pattern offloads computational tasks to Python or Bash scripts in the `scripts/` directory. The skill prompt tells Claude to execute the script and process its output.

SKILL.md example:

```
Run scripts/analyzer.py on the target directory:

`python {baseDir}/scripts/analyzer.py --path "$USER_PATH" --outp

Parse the generated `report.json` and present findings.
```

Required tools:

```
allowed-tools: "Bash(python {baseDir}/scripts/*:*), Read, Write"
```

Pattern 2: Read - Process - Write

Use case: File transformation and data processing.

The simplest pattern — read input, transform it following instructions, write output. Useful for format conversions, data cleanup, or report generation.

SKILL.md example:

```
### Processing Workflow
1. Read input file using Read tool
2. Parse content according to format
3. Transform data following specifications
4. Write output using Write tool
5. Report completion with summary
```

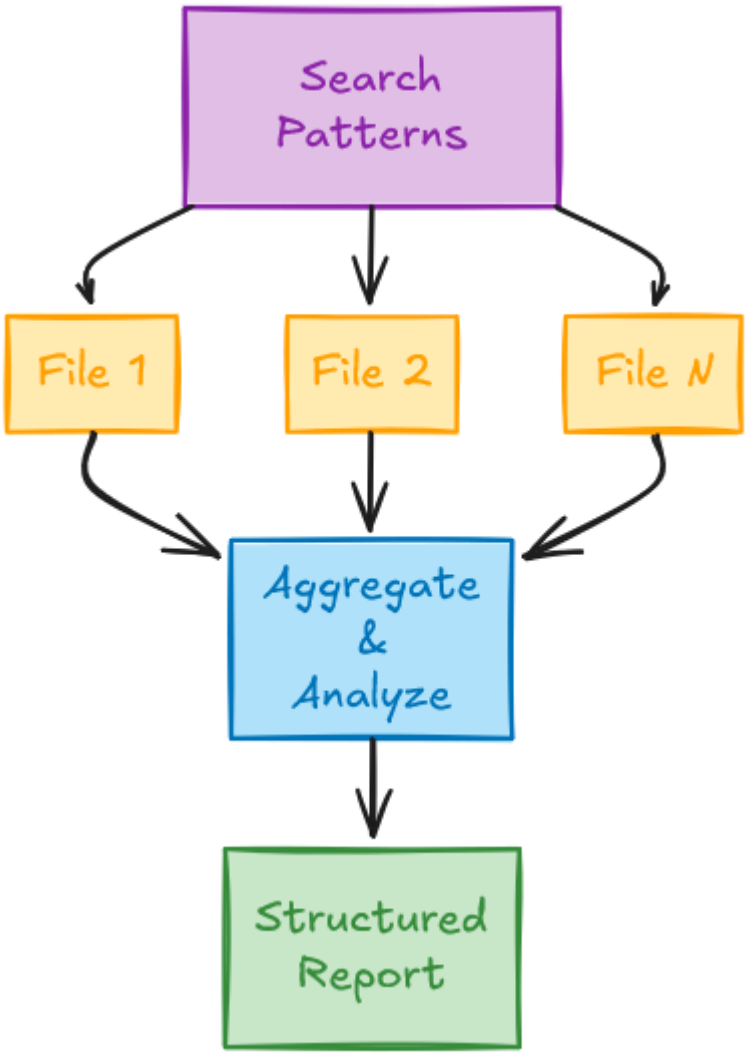
Required tools:

```
allowed-tools: "Read, Write"
```

Pattern 3: Search - Analyze - Report

Use case: Codebase analysis and pattern detection.

Search the codebase for patterns using Grep, read matching files for context, analyze findings, and generate a structured report. Or, search enterprise data store for data, analyze the retrieved data for information, and generate a structured report.



SKILL.md example:

```
### Analysis Process
1. Use Grep to find relevant code patterns
2. Read each matched file
3. Analyze for vulnerabilities
4. Generate structured report
```

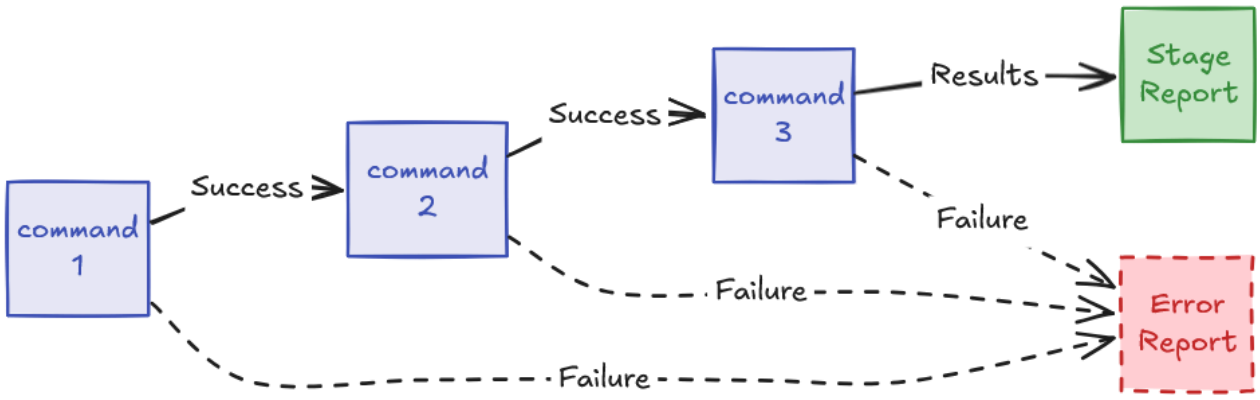
Required tools:

```
allowed-tools: "Grep, Read"
```

Pattern 4: Command Chain Execution

Use case: Multi-step operations with dependencies.

Execute a sequence of commands where each step depends on the previous one's success. Common for CI/CD-like workflows.



SKILL.md example:

```
Execute analysis pipeline:
npm install && npm run lint && npm test

Report results from each stage.
```

Required tools:

```
allowed-tools: "Bash(npm install:*), Bash(npm run:*), Read"
```

Advanced Patterns

Wizard-Style Multi-Step Workflows

Use case: Complex processes requiring user input at each step.

Break complex tasks into discrete steps with explicit user confirmation between each phase. Useful for setup wizards, configuration tools, or guided processes.

SKILL.md example:

```
### Workflow

#### Step 1: Initial Setup
1. Ask user for project type
2. Validate prerequisites exist
```

```
3. Create base configuration
Wait for user confirmation before proceeding.

#### Step 2: Configuration
1. Present configuration options
2. Ask user to choose settings
3. Generate config file
Wait for user confirmation before proceeding.

#### Step 3: Initialization
1. Run initialization scripts
2. Verify setup successful
3. Report results
```

Template-Based Generation

Use case: Creating structured outputs from templates stored in `assets/`.

Load templates, fill placeholders with user-provided or generated data, and write the result. Common for report generation, boilerplate code creation, or documentation.

SKILL.md example:

```
### Generation Process
1. Read template from {baseDir}/assets/template.html
2. Parse user requirements
3. Fill template placeholders:
   -  → user-provided name
   -  → generated summary
   -  → current date
4. Write filled template to output file
5. Report completion
```

Iterative Refinement

Use case: Processes requiring multiple passes with increasing depth.

Perform broad analysis first, then progressively deeper dives on identified issues. Useful for code review, security audits, or quality analysis.

SKILL.md example:

```
### Iterative Analysis

#### Pass 1: Broad Scan
1. Search entire codebase for patterns
2. Identify high-level issues
3. Categorize findings

#### Pass 2: Deep Analysis
For each high-level issue:
1. Read full file context
2. Analyze root cause
3. Determine severity

#### Pass 3: Recommendation
For each finding:
1. Research best practices
2. Generate specific fix
3. Estimate effort

Present final report with all findings and recommendations.
```

Context Aggregation

Use case: Combining information from multiple sources to build comprehensive understanding.

Gather data from different files and tools, synthesize into a coherent picture. Useful for project summaries, dependency analysis, or impact assessments.

SKILL.md example:

```
### Context Gathering
1. Read project README.md for overview
2. Analyze package.json for dependencies
3. Grep codebase for specific patterns
4. Check git history for recent changes
5. Synthesize findings into coherent summary
```

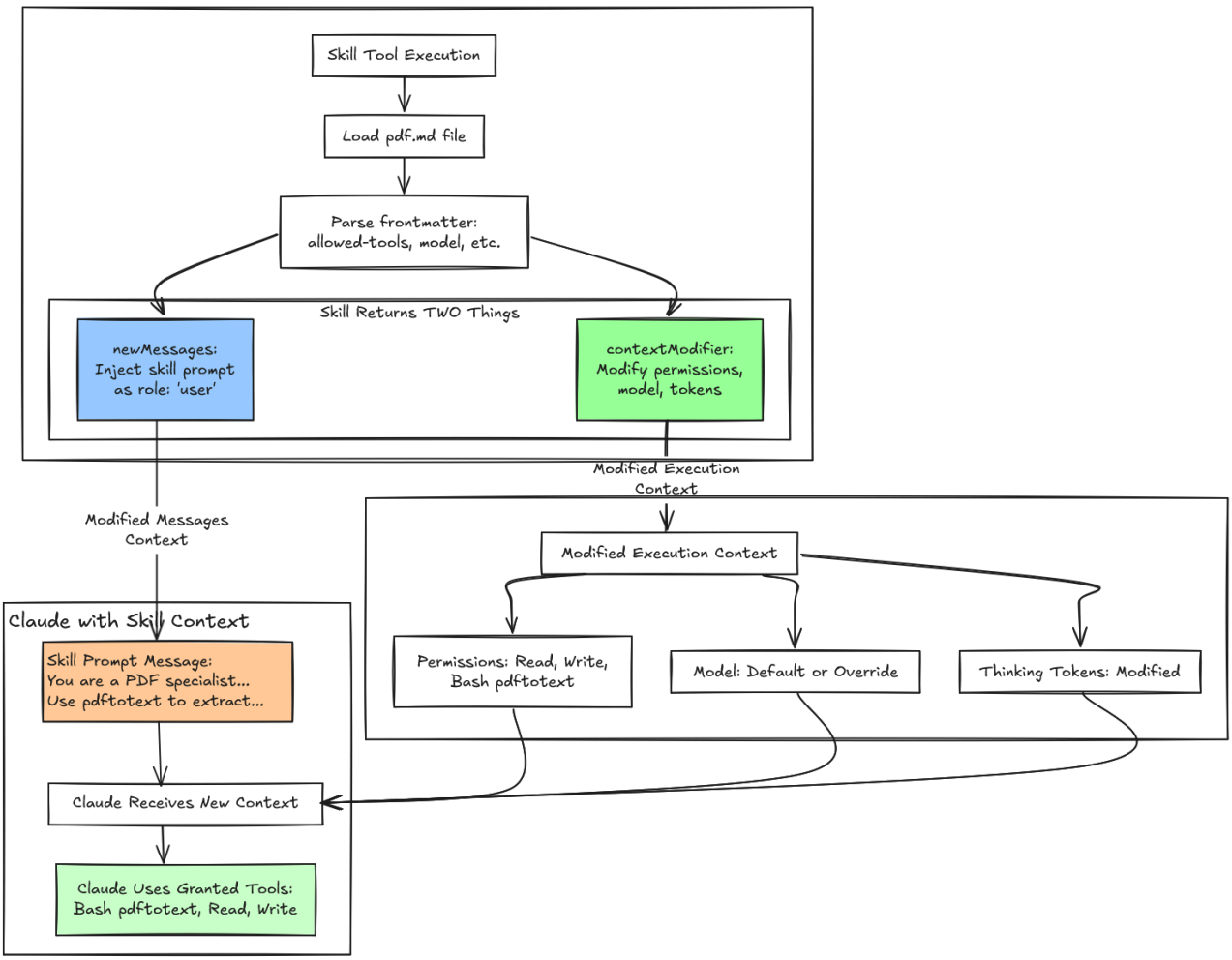
Agent Skills Internal Architecture

With the overview and building process covered, we can now examine how skills actually work under the hood. The skills system operates through a meta-tool architecture where a tool named `Skill` acts as a container and dispatcher for all individual skills. This design fundamentally distinguishes skills from traditional tools in both implementation and purpose.

> The `Skill` tool is a meta-tool that manages all skills

Skills Object Design

Traditional tools like `Read`, `Bash`, or `Write` execute discrete actions and return immediate results. Skills operate differently. Rather than performing actions directly, they inject specialized instructions into the conversation history and dynamically modify Claude's execution environment. This happens through two user messages—one containing metadata visible to users, another containing the full skill prompt hidden from the UI but sent to Claude - and by altering the agent's context to change permissions, switch models, and adjust thinking token parameters for the duration of the skill's use.



Feature	Normal Tool	Skill Tool
Essence	Direct action executor	Prompt injection + context modifier
Message Role	assistant → tool_use user → tool_result	assistant → tool_use Skill user → tool_result user → skill prompt ← INJECTED!
Complexity	Simple (3-4 messages)	Complex (5-10+ messages)
Context	Static	Dynamic (modified per turn)
Persistence	Tool interactions only	Tool interactions + skill prompts
Token Overhead	Minimal (~100 tokens)	Significant (~1,500+ tokens per turn)
Use Case	Simple, direct tasks	Complex, guided workflows

The complexity is substantial. Normal tools generate simple message exchanges—an assistant tool call followed by a user result. Skills inject multiple messages, operate within a dynamically modified context, and carry significant token overhead to provide the specialized instructions that guide Claude's behavior.

Understanding how the `Skill` meta-tool works reveals the mechanics of this system. Let's examine its structure:

```
Pd = {
  name: "Skill",  // The tool name constant: $N = "Skill"

  inputSchema: {
    command: string  // E.g., "pdf", "skill-creator"
  },

  outputSchema: {
    success: boolean,
    commandName: string
  },

  // 🔑 KEY FIELD: This generates the skills list
  prompt: async () ⇒ fN2(),

  // Validation and execution
  validateInput: async (input, context) ⇒ { /* 5 error codes */
  checkPermissions: async (input, context) ⇒ { /* allow/deny/as
```

```
  call: async *(input, context) ⇒ { /* yields messages + contex
}
```

The `prompt` field distinguishes the Skill tool from other tools like `Read` or `Bash`, which have static descriptions. Instead of a fixed string, the Skill tool uses a dynamic prompt generator that constructs its description at runtime by aggregating the names and descriptions of all available skills. This implements **progressive disclosure** — the system loads only the minimal metadata (skill names and descriptions from frontmatter) into Claude's initial context, providing just enough information for the model to decide which skill matches the user's intent. The full skill prompt loads only after Claude makes that selection, preventing context bloat while maintaining discoverability.

```
async function fN2() {
  let A = await atA(),
    {
      modeCommands: B,
      limitedRegularCommands: Q
    } = vN2(A),
    G = [...B, ...Q].map((W) ⇒ W.userFacingName()).join(", ");
  l(`Skills and commands included in Skill tool: ${G}`);
  let Z = A.length - B.length,
    Y = nS6(B),
    J = aS6(Q, Z);
  return `Execute a skill within the main conversation

<skills_instructions>
When users ask you to perform tasks, check if any of the availab

How to use skills:
- Invoke skills using this tool with the skill name only (no arg
- When you invoke a skill, you will see <command-message>The "{n
- The skill's prompt will expand and provide detailed instructio
- Examples:
  - \`command: "pdf"\` - invoke the pdf skill
  - \`command: "xlsx"\` - invoke the xlsx skill
  - \`command: "ms-office-suite:pdf"\` - invoke using fully qual

Important:
- Only use skills listed in <available_skills> below
```

```
- Do not invoke a skill that is already running
- Do not use this tool for built-in CLI commands (like /help, /c
</skills_instructions>

<available_skills>
${Y}${J}
</available_skills>
`;
}
```

Unlike how some tools lives in the system prompts for certain assistants such as ChatGPT, Claude **agent skills do not live in the system prompt**. They live in the `tools` array as part of the `Skill` tool's description. Names of the individual skills is represented as part of the `Skill` meta-tool's input schema's `command` field. To better visualize how it looks, here's the actual API request structure:

```
{
  "model": "claude-sonnet-4-5-20250929",
  "system": "You are Claude Code, Anthropic's official CLI...",
  "messages": [
    {"role": "user", "content": "Help me create a new skill"},
    // ... conversation history
  ],
  "tools": [  // ← Tools array sent to Claude
    {
      "name": "Skill",  // ← The meta-tool
      "description": "Execute a skill...\n\n<skills_instructions
      "input_schema": {
        "type": "object",
        "properties": {
          "command": {
            "type": "string",
            "description": "The skill name (no arguments)"  // ←
          }
        }
      }
    },
    {
      "name": "Bash",
      "description": "Execute bash commands...",
```

```
      // ...
    },
    {
      "name": "Read",
      // ...
    }
    // ... other tools
  ]
}
```

The `<available_skills>` section lives within the Skill tool's description and gets regenerated for each API request. The system dynamically builds this list by aggregating currently loaded skills from user and project configurations, plugin-provided skills, and any built-in skills, subject to a token budget limit of 15,000 characters by default. This budget constraint forces skill authors to write concise descriptions and ensures the tool description doesn't overwhelm the model's context window.

Skill Conversation and Execution Context Injection Design

Most LLM APIs supports `role: "system"` messages that could theoretically carry system prompts. In fact, OpenAI's ChatGPT carries its default tools in its system prompts, including `bio` for memory, `automations` for task scheduling, `canmore` for controlling canvas, `img_gen` for image generation, `file_search`, `python`, and `web` for Internet search. And at the end, the tools prompt takes up around 90% of the token counts in its system prompt. This could be useful but hardly efficient if we have lots of tools and/or skills to be loaded into the context.

However, system messages have different semantics that make them unsuitable for skills. System messages set global context that persists across the entire conversation, affecting all subsequent turns with higher authority than user instructions.

Skills need temporary, scoped behavior. The `skill-creator` skill should only affect skill creating related tasks, not transform Claude into a permanent PDF specialist for the rest of the session. Using `role: "user"` with `isMeta: true` makes the skill prompt appear as user input to Claude, keeping it temporary and localized to the current interaction. After the skill completes, the conversation

returns to normal conversation context and execution context without residual behavioral modifications.

Normal tools like `Read`, `Write`, or `Bash` have simple communication patterns. When Claude invokes `Read`, it sends a file path, receives the file contents, and continues working. The user sees "Claude used the Read tool" in their transcript, and that's sufficient transparency. The tool did one thing, returned a result, and that's the end of the interaction. Skills operate fundamentally differently. Instead of executing discrete actions and returning results, skills inject comprehensive instruction sets that modify how Claude reasons about and approaches the task. This creates a design challenge that normal tools never face: users need transparency about which skills are running and what they're doing, while Claude needs detailed, potentially verbose instructions to execute the skill correctly. If users see the full skill prompts in their chat transcript, the UI becomes cluttered with thousands of words of internal AI instructions. If the skill activation is completely hidden, users lose visibility into what the system is doing on their behalf. The solution requires separating these two communication channels into distinct messages with different visibility rules.

The skills system uses an `isMeta` flag on each message to control whether it appears in the user interface. When `isMeta: false` (or when the flag is omitted and defaults to false), the message renders in the conversation transcript that users see. When `isMeta: true`, the message gets sent to the Anthropic API as part of Claude's conversation context but never appears in the UI. This simple boolean flag enables sophisticated dual-channel communication: one stream for human users, another for the AI model. Meta-prompting for meta-tools!

When a skill executes, the system injects two separate user messages into the conversation history. The first carries skill metadata with `isMeta: false`, making it visible to users as a status indicator. The second carries the full skill prompt with `isMeta: true`, hiding it from the UI while making it available to Claude. This split solves the transparency vs clarity tradeoff by showing users what's happening without overwhelming them with implementation details.

The metadata message uses a concise XML structure that the frontend can parse and display appropriately:

```
let metadata = [
  `<command-message>${statusMessage}</command-message>`,
  `<command-name>${skillName}</command-name>`,
```

```
  args ? `<command-args>${args}</command-args>` : null
].filter(Boolean).join('\n');

// Message 1: NO isMeta flag → defaults to false → VISIBLE
messages.push({
  content: metadata,
  autocheckpoint: checkpointFlag
});
```

When the PDF skill activates, for example, users see a clean loading indicator in their transcript:

```
<command-message>The "pdf" skill is loading</command-message>
<command-name>pdf</command-name>
<command-args>report.pdf</command-args>
```

This message stays intentionally minimal - typically 50 to 200 characters. The XML tags enable the frontend to render it with special formatting, validate that proper `<command-message>` tags are present, and maintain an audit trail of which skills executed during the session. Because the `isMeta` flag defaults to false when omitted, this metadata automatically appears in the UI.

The skill prompt message takes the opposite approach. It loads the full content from `SKILL.md`, potentially augments it with additional context, and explicitly sets `isMeta: true` to hide it from users:

```
let skillPrompt = await skill.getPromptForCommand(args, context)

// Augment with prepend/append content if needed
let fullPrompt = prependContent.length > 0 || appendContent.leng
  ? [...prependContent, ...appendContent, ...skillPrompt]
  : skillPrompt;

// Message 2: Explicit isMeta: true → HIDDEN
messages.push({
  content: fullPrompt,
  isMeta: true  // HIDDEN FROM UI, SENT TO API
});
```

A typical skill prompt runs 500 to 5,000 words and provides comprehensive guidance to transform Claude's behavior. The PDF skill prompt might contain:

```
You are a PDF processing specialist.

Your task is to extract text from PDF documents using the pdftot

### Process

1. Validate the PDF file exists
2. Run pdftotext command to extract text
3. Read the output file
4. Present the extracted text to the user

### Tools Available

You have access to:
- Bash(pdftotext:*) - For running pdftotext command
- Read - For reading extracted text
- Write - For saving results if needed

### Output Format

Present the extracted text clearly formatted.

Base directory: /path/to/skill
User arguments: report.pdf
```

This prompt establishes task context, outlines the workflow, specifies available tools, defines output format, and provides environment-specific paths. The markdown structure with headers, lists, and code blocks helps Claude parse and follow the instructions. With `isMeta: true`, this entire prompt gets sent to the API but never clutters the user's transcript.

Beyond the core metadata and skill prompt, skills can inject additional conditional messages for attachments and permissions:

```
let allMessages = [
  createMessage({ content: metadata, autocheckpoint: flag }),   /
  createMessage({ content: skillPrompt, isMeta: true }),        /
```

```
  ...attachmentMessages,                                          /
  ...(allowedTools.length || skill.model ? [
    createPermissionsMessage({                                    /
      type: "command_permissions",
      allowedTools: allowedTools,
      model: skill.useSmallFastModel ? getFastModel() : skill.mo
    })
  ] : [])
];
```

Attachment messages can carry diagnostics information, file references, or additional context that supplements the skill prompt. Permission messages only appear when the skill specifies `allowed-tools` in its frontmatter or requests a model override, providing metadata that modifies the runtime execution environment. This modular composition allows each message to have a specific purpose and be included or excluded based on the skill's configuration, extending the basic two-message pattern to handle more complex scenarios while maintaining the same visibility control through `isMeta` flags.

Why Two Messages Instead of One?

A single-message design would force an impossible choice. Setting `isMeta: false` would make the entire message visible, dumping thousands of words of AI instructions into the user's chat transcript. Users would see something like:

```
┌──────────────────────────────────────────────────┐
│ The "pdf" skill is loading                       │
│                                                  │
│ You are a PDF processing specialist.             │
│                                                  │
│ Your task is to extract text from PDF            │
│ documents using the pdftotext tool.              │
│                                                  │
│ ## Process                                       │
│                                                  │
│ 1. Validate the PDF file exists                  │
│ 2. Run pdftotext command to extract text         │
│ 3. Read the output file                          │
│ ... [500 more lines] ...                         │
└──────────────────────────────────────────────────┘
```

The UI becomes unusable, filled with internal implementation details meant for Claude, not humans. Alternatively, setting `isMeta: true` would hide everything, providing no transparency about which skill activated or what arguments it received. Users would have no visibility into what the system is doing on their behalf.

The two-message split resolves this by giving each message a different `isMeta` value. Message 1 with `isMeta: false` provides user-facing transparency. Message 2 with `isMeta: true` provides Claude with detailed instructions. This granular control enables transparency without information overload.

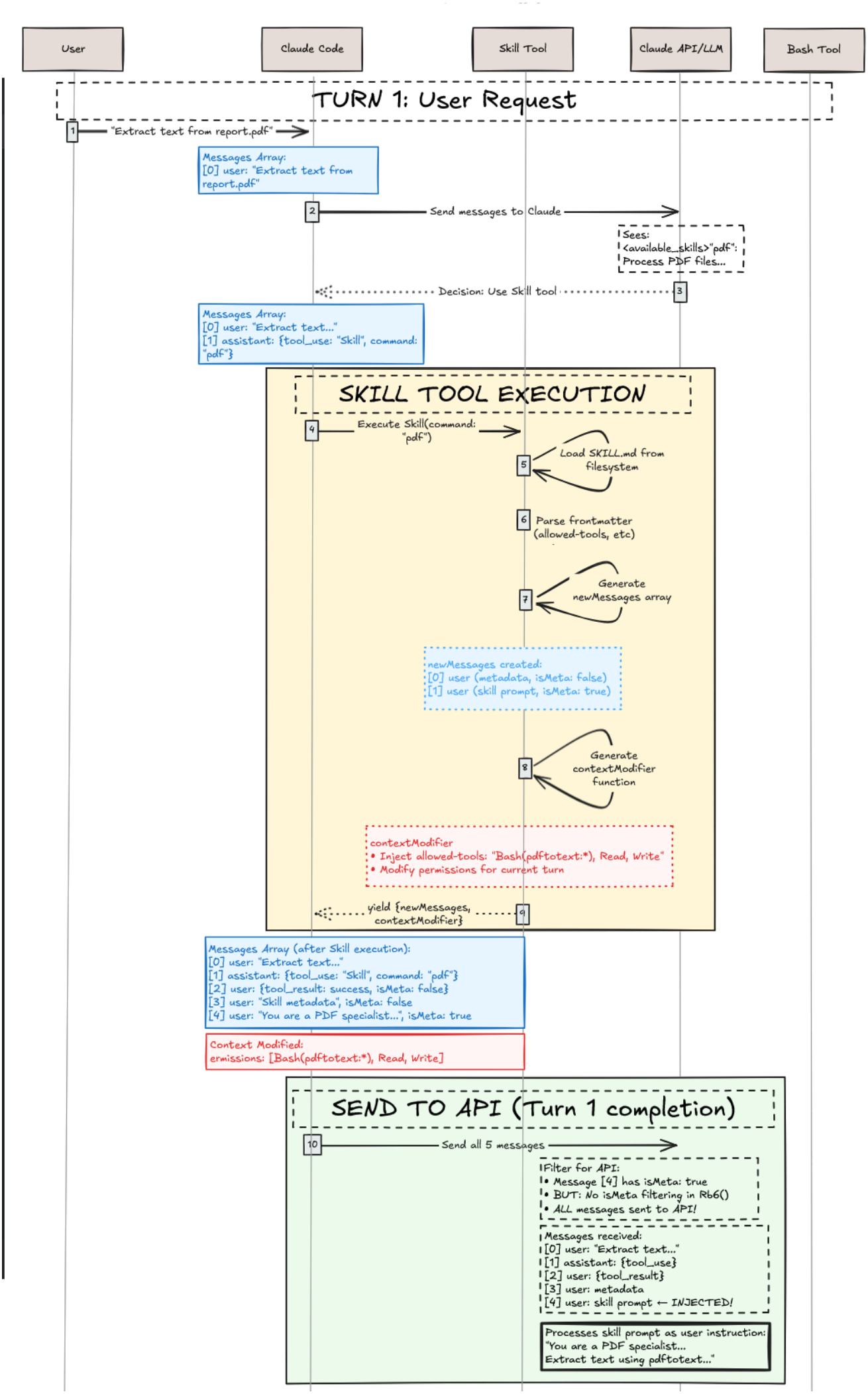
The messages also serve fundamentally different audiences and purposes:

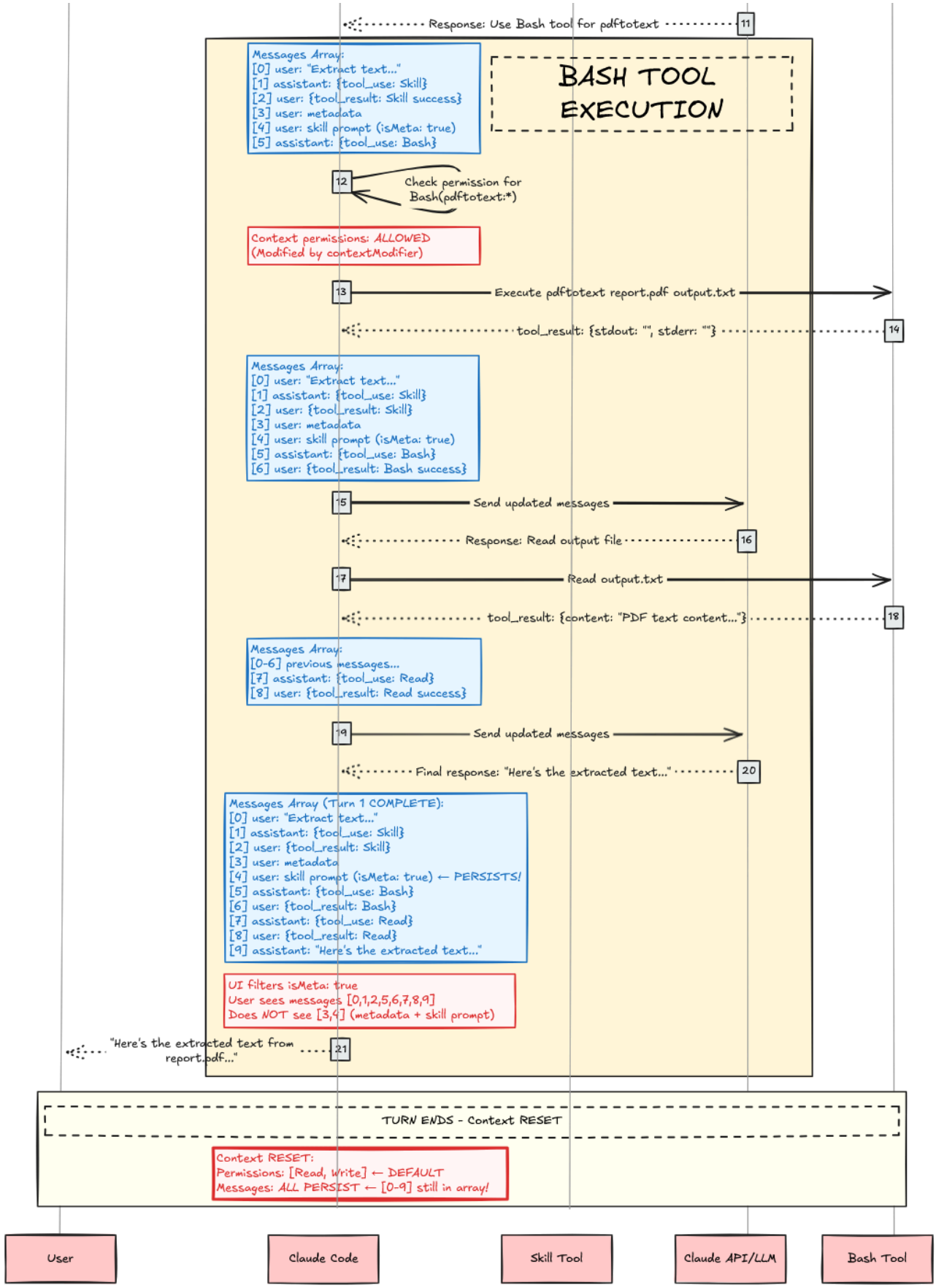
Aspect	Metadata Message	Skill Prompt Message
Audience	Human user	Claude (AI)
Purpose	Status/transparency	Instructions/guidance
Length	~50-200 chars	~500-5,000 words
Format	Structured XML	Natural language markdown
Visibility	Should be visible	Should be hidden
Content	"What is happening?"	"How to do it?"

The codebase even processes these messages through different paths. The metadata message gets parsed for `<command-message>` tags, validated, and formatted for UI display. The skill prompt message gets sent directly to the API without parsing or validation—it's raw instructional content meant only for Claude's reasoning process. Combining them would violate the Single Responsibility Principle by forcing one message to serve two distinct audiences through two different processing pipelines.

Case Study: Execution Lifecycle

Now covered Agent Skills internal architecture, let's walk through what happens when a user says "Extract text from report.pdf" by examining the complete execution flow using a hypothetical `pdf` skill as a case study.


User
Claude Code
Skill Tool
Claude API/LLM
Bash Tool
TURN 1: User Request
1
"Extract text from report.pdf"
Messages Array:
[0] user: "Extract text from report.pdf"
2
Send messages to Claude
Sees:
<available_skills>"pdf":
Process PDF files...
Decision: Use Skill tool
3
Messages Array:
[0] user: "Extract text..."
[1] assistant: {tool_use: "Skill", command: "pdf"}
SKILL TOOL EXECUTION
4
Execute Skill(command: "pdf")
5
Load SKILL.md from filesystem
6
Parse frontmatter (allowed-tools, etc)
7
Generate newMessages array
newMessages created:
[0] user (metadata, isMeta: false)
[1] user (skill prompt, isMeta: true)
8
Generate contextModifier function
contextModifier
• Inject allowed-tools: "Bash(pdftotext:*), Read, Write"
• Modify permissions for current turn
yield {newMessages, contextModifier}
9
Messages Array (after Skill execution):
[0] user: "Extract text..."
[1] assistant: {tool_use: "Skill", command: "pdf"}
[2] user: {tool_result: success, isMeta: false}
[3] user: "Skill metadata", isMeta: false
[4] user: "You are a PDF specialist...", isMeta: true
Context Modified:
ermissions: [Bash(pdftotext:*), Read, Write]
SEND TO API (Turn 1 completion)
10
Send all 5 messages
Filter for API:
• Message [4] has isMeta: true
• BUT: No isMeta filtering in Rb6()
• ALL messages sent to API!
Messages received:
[0] user: "Extract text..."
[1] assistant: {tool_use}
[2] user: {tool_result}
[3] user: metadata
[4] user: skill prompt ← INJECTED!
Processes skill prompt as user instruction:
"You are a PDF specialist...
Extract text using pdftotext..."



Phase 1: Discovery & Loading (Startup)

When Claude Code starts, it scans for skills:

```
async function getAllCommands() {
  // Load from all sources in parallel
```

```
  let [userCommands, skillsAndPlugins, pluginCommands, builtins]
    await Promise.all([
      loadUserCommands(),       // ~/.claude/commands/
      loadSkills(),             // .claude/skills/ + plugins
      loadPluginCommands(),     // Plugin-defined commands
      getBuiltinCommands()      // Hardcoded commands
    ]);

  return [...userCommands, ...skillsAndPlugins, ...pluginCommand
    .filter(cmd => cmd.isEnabled());
}
```

```
// Specific skill loading
async function loadPluginSkills(plugin) {
  // Check if plugin has skills
  if (!plugin.skillsPath) return [];

  // Two patterns supported:
  // 1. Root SKILL.md in skillsPath
  // 2. Subdirectories with SKILL.md

  const skillFiles = findSkillMdFiles(plugin.skillsPath);
  const skills = [];

  for (const file of skillFiles) {
    const content = readFile(file);
    const { frontmatter, markdown } = parseFrontmatter(content);

    skills.push({
      type: "prompt",
      name: `${plugin.name}:${getSkillName(file)}`,
      description: `${frontmatter.description} (plugin:${plugin.
      whenToUse: frontmatter.when_to_use,  // ← Note: underscore
      allowedTools: parseTools(frontmatter['allowed-tools']),
      model: frontmatter.model === "inherit" ? undefined : front
      isSkill: true,
      promptContent: markdown,
      // ... other fields
    });
  }

  return skills;
}
```

For the pdf skill, this produces:

```
{
  type: "prompt",
  name: "pdf",
  description: "Extract text from PDF documents (plugin:document
  whenToUse: "When user wants to extract or process text from PD
  allowedTools: ["Bash(pdftotext:*)", "Read", "Write"],
  model: undefined,  // Uses session model
```

```
  isSkill: true,
  disableModelInvocation: false,
  promptContent: "You are a PDF processing specialist...",
  // ... other fields
}
```

Phase 2: Turn 1 - User Request & Skill Selection

The user sends a request: “Extract text from report.pdf”. Claude receives this message along with the `Skill` tool in its tools array. Before Claude can decide to invoke the pdf skill, the system must present available skills in the Skill tool’s description.

Skill Filtering & Presentation

Not all loaded skills appear in the Skill tool. A skill MUST have either `description` OR `when_to_use` in frontmatter, or it’s filtered out. Filtering criteria:

```
async function getSkillsForSkillTool() {
  const allCommands = await getAllCommands();

  return allCommands.filter(cmd =>
    cmd.type === "prompt" &&
    cmd.isSkill === true &&
    !cmd.disableModelInvocation &&
    (cmd.source !== "builtin" || cmd.isModeCommand === true) &&
    (cmd.hasUserSpecifiedDescription || cmd.whenToUse)  // ← Mus
  );
}
```

Skill Formatting

Each skill is formatted for the `<available_skills>` section. As an example, our hypothetical `pdf` skill could be formatted into `"pdf": Extract text from PDF documents - When user wants to extract or process text from PDF files`

```
function formatSkill(skill) {
  let name = skill.name;
  let description = skill.whenToUse
```

```
    ? `${skill.description} - ${skill.whenToUse}`
    : skill.description;

  return `"${name}": ${description}`;
}
```

Claude's Decision Process

Now, when user prompts: "Extract text from report.pdf". Claude receives the API request with the `Skill` tool, reads the `<available_skills>`, and reasons (hypothetically, as we do not see reasoning traces):

```
Internal reasoning:
- User wants to "extract text from report.pdf"
- This is a PDF processing task
- Looking at available skills...
- "pdf": Extract text from PDF documents - When user wants to ex
- This matches! The user wants to extract text from a PDF
- Decision: Invoke Skill tool with command="pdf"
```

Note that there's no algorithmic matching here. No lexical matching. No semantic matching. No searches. This is pure LLM reasoning for its decisions based on the description of the skill. Once done, Claude returns a tool use:

```
{
  "type": "tool_use",
  "id": "toolu_123abc",
  "name": "Skill",
  "input": {
    "command": "pdf"
  }
}
```

Phase 3: Skill Tool Execution

The Skill tool now executes. This corresponds to the yellow "SKILL TOOL EXECUTION" box in the sequence diagram, which performs validation, permission checks, file loading, and context modification before yielding the result.

Step 1: Validation

```
async validateInput({ command }, context) {
  let skillName = command.trim().replace(/^\//, "");

  // Error 1: Empty
  if (!skillName) return { result: false, errorCode: 1 };

  // Error 2: Unknown skill
  const allSkills = await getAllCommands();
  if (!skillExists(skillName, allSkills)) {
    return { result: false, errorCode: 2 };
  }

  // Error 3: Can't load
  const skill = getSkill(skillName, allSkills);
  if (!skill) return { result: false, errorCode: 3 };

  // Error 4: Model invocation disabled
  if (skill.disableModelInvocation) {
    return { result: false, errorCode: 4 };
  }

  // Error 5: Not prompt-based
  if (skill.type !== "prompt") {
    return { result: false, errorCode: 5 };
  }

  return { result: true };
}
```

The pdf skill passes all validation checks ✓

Step 2: Permission Check

```
async checkPermissions({ command }, context) {
  const skillName = command.trim().replace(/^\//, "");
  const permContext = (await context.getAppState()).toolPermissi

  // Check deny rules
  for (const [pattern, rule] of getDenyRules(permContext)) {
    if (matches(skillName, pattern)) {
      return { behavior: "deny", message: "Blocked by permissior
```

```
    }
  }

  // Check allow rules
  for (const [pattern, rule] of getAllowRules(permContext)) {
    if (matches(skillName, pattern)) {
      return { behavior: "allow" };
    }
  }

  // Default: ask user
  return { behavior: "ask", message: `Execute skill: ${skillName
}
```

Assuming no rules, user is prompted: “Execute skill: pdf?”
User approves ✓

Step 3: Load Skill File and Generate Execution Context Modification

With validation and permissions approved, the Skill tool loads the skill file and prepares the execution context modification:

```
async *call({ command }, context) {
  const skillName = command.trim().replace(/^\//, "");
  const allSkills = await getAllCommands();
  const skill = getSkill(skillName, allSkills);

  // Load the skill prompt
  const promptContent = await skill.getPromptForCommand("", cont

  // Generate metadata tags
  const metadata = [
    `<command-message>The "${skill.userFacingName()}" skill is l
    `<command-name>${skill.userFacingName()}</command-name>`
  ].join('\n');

  // Create messages
  const messages = [
    { type: "user", content: metadata },  // Visible to user
    { type: "user", content: promptContent, isMeta: true },  //
    // ... attachments, permissions
  ];
```

```
// Extract configuration
const allowedTools = skill.allowedTools || [];
const modelOverride = skill.model;

// Yield result with execution context modifier
yield {
  type: "result",
  data: { success: true, commandName: skillName },
  newMessages: messages,

  // 🔑 Execution context modification function
  contextModifier(context) {
    let modified = context;

    // Inject allowed tools
    if (allowedTools.length > 0) {
      modified = {
        ...modified,
        async getAppState() {
          const state = await context.getAppState();
          return {
            ...state,
            toolPermissionContext: {
              ...state.toolPermissionContext,
              alwaysAllowRules: {
                ...state.toolPermissionContext.alwaysAllowRule
                command: [
                  ...state.toolPermissionContext.alwaysAllowRu
                  ...allowedTools  // ← Pre-approve these tool
                ]
              }
            }
          };
        }
      };
    }

    // Override model
    if (modelOverride) {
      modified = {
```

```
            ...modified,
            options: {
              ...modified.options,
              mainLoopModel: modelOverride
            }
          };
        }

        return modified;
      }
    };
  }
```

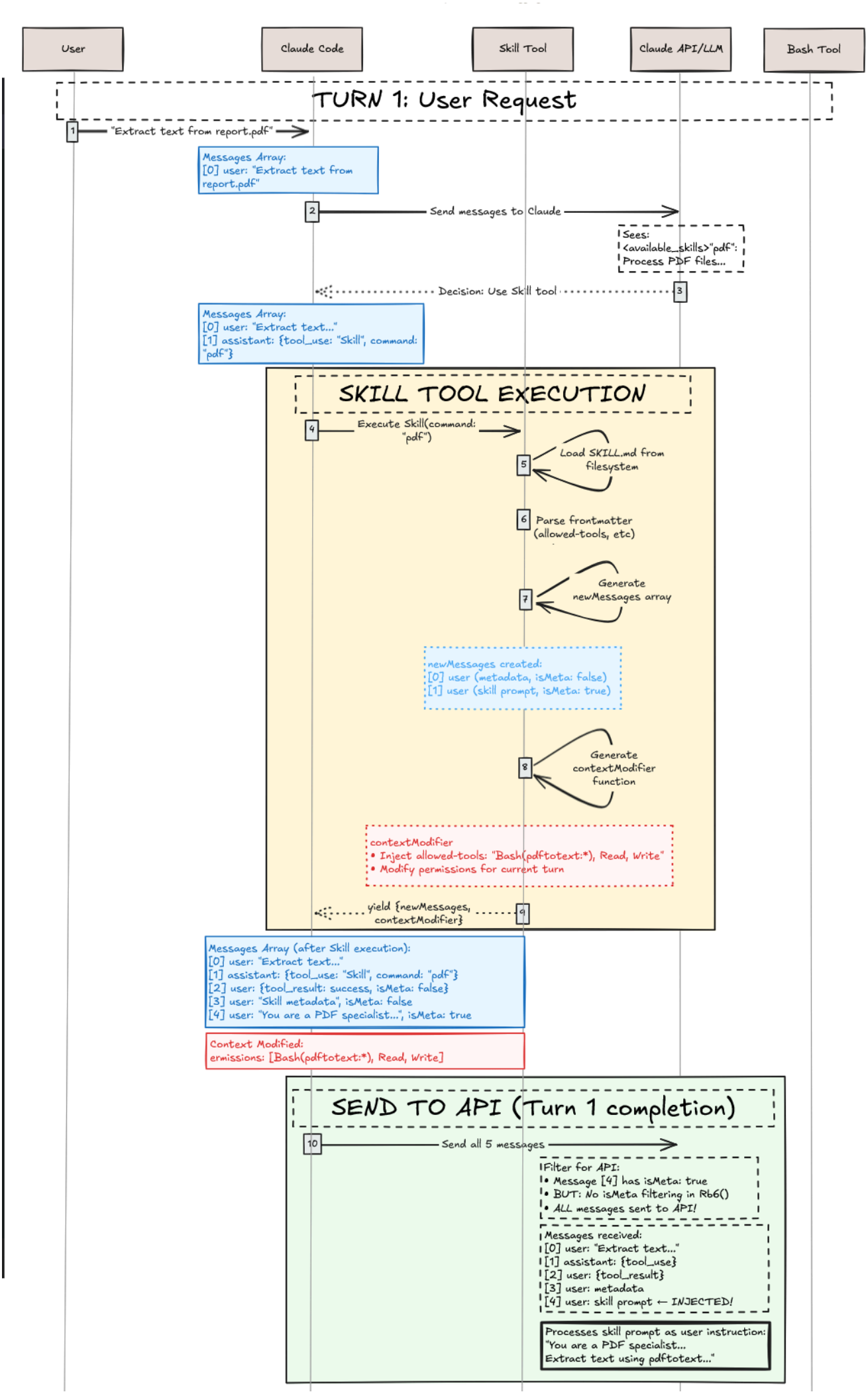
The Skill tool yields its result containing `newMessages` (metadata + skill prompt + permissions for conversation context injection) and `contextModifier` (tool permissions + model override for execution context modification). This completes the yellow "SKILL TOOL EXECUTION" box from the sequence diagram.

Phase 4: Send to API (Turn 1 Completion)

The system constructs the complete messages array to send to the Anthropic API. This includes all messages from the conversation plus the newly injected skill messages:

```
// Complete message array sent to API for Turn 1
{
  model: "claude-sonnet-4-5-20250929",
  messages: [
    {
      role: "user",
      content: "Extract text from report.pdf"
    },
    {
      role: "assistant",
      content: [
        {
          type: "tool_use",
          id: "toolu_123abc",
          name: "Skill",
          input: { command: "pdf" }
```

```
        }
      ]
    },
    {
      role: "user",
      content: "<command-message>The \"pdf\" skill is loading</c
      // isMeta: false (default) - VISIBLE to user in UI
    },
    {
      role: "user",
      content: "You are a PDF processing specialist...\n\n## Pro
      isMeta: true  // HIDDEN from UI, sent to API
    },
    {
      role: "user",
      content: {
        type: "command_permissions",
        allowedTools: ["Bash(pdftotext:*)", "Read", "Write"],
        model: undefined
      }
    }
  ]
}
```


User
Claude Code
Skill Tool
Claude API/LLM
Bash Tool
TURN 1: User Request
1
"Extract text from report.pdf"
Messages Array:
[0] user: "Extract text from report.pdf"
2
Send messages to Claude
Sees:
<available_skills>"pdf":
Process PDF files...
Decision: Use Skill tool
3
Messages Array:
[0] user: "Extract text..."
[1] assistant: {tool_use: "Skill", command: "pdf"}
SKILL TOOL EXECUTION
4
Execute Skill(command: "pdf")
5
Load SKILL.md from filesystem
6
Parse frontmatter (allowed-tools, etc)
7
Generate newMessages array
newMessages created:
[0] user (metadata, isMeta: false)
[1] user (skill prompt, isMeta: true)
8
Generate contextModifier function
contextModifier
• Inject allowed-tools: "Bash(pdftotext:*), Read, Write"
• Modify permissions for current turn
yield {newMessages, contextModifier}
9
Messages Array (after Skill execution):
[0] user: "Extract text..."
[1] assistant: {tool_use: "Skill", command: "pdf"}
[2] user: {tool_result: success, isMeta: false}
[3] user: "Skill metadata", isMeta: false
[4] user: "You are a PDF specialist...", isMeta: true
Context Modified:
ermissions: [Bash(pdftotext:*), Read, Write]
SEND TO API (Turn 1 completion)
10
Send all 5 messages
Filter for API:
• Message [4] has isMeta: true
• BUT: No isMeta filtering in Rb6()
• ALL messages sent to API!
Messages received:
[0] user: "Extract text..."
[1] assistant: {tool_use}
[2] user: {tool_result}
[3] user: metadata
[4] user: skill prompt ← INJECTED!
Processes skill prompt as user instruction:
"You are a PDF specialist...
Extract text using pdftotext..."

The above shows what we have done up to this point. The execution context modifier is applied, pre-approving `Bash(pdftotext:*)`, `Read`, and `Write` for subsequent tool calls. The request is sent to the Anthropic API. This concludes the skill execution. If this is a normal tool, we are all done. However, skills are different. Agent skill has only injected the conversation context and execution contexts. THis means we still need to invoke Claude agent with all these injected contexts to complete user's request!

Phase 5: Bash Tool Execution (Tool use with Skill Context!)

Claude receives the API response with the injected conversation context. The skill prompt has transformed Claude's behavior, providing it with:

- Specialized PDF processing instructions (conversation context)
- Pre-approved access to `Bash(pdftotext:*)`, `Read`, and `Write` tools (execution context)
- A clear workflow to follow (conversation context)

Claude processes the context and follows the pdf skill's workflow:

```
I'll extract text from report.pdf. Let me process the file.

[Following pdf skill's instructions]
1. Validate that report.pdf exists
2. Run pdftotext command to extract text
3. Read the output file
4. Present the extracted text to you
```

Claude uses the Bash tool (pre-approved, no user prompt required):

```
{
  "type": "tool_use",
  "id": "toolu_456def",
  "name": "Bash",
  "input": {
    "command": "pdftotext report.pdf output.txt",
    "description": "Extract text from PDF using pdftotext"
  }
}
```

The Bash tool executes successfully, returning the result. Claude then reads the output file using the Read tool and presents the extracted text to the user. The skill has successfully guided Claude through the specialized PDF extraction workflow by injecting instructions into the conversation context and modifying the execution context for tool permissions.

Conclusion: The Mental Model Recap

Skills in Claude Code are **prompt-based conversation and execution context modifiers** that work through a meta-tool architecture:

Key Takeaways:

1. Skills are **prompt templates** in `SKILL.md` files, not executable code
2. The **Skill tool** (capital S) is a meta-tool in the `tools` array that manages individual skills, not in the system prompt
3. Skills **modify conversation context** by injecting instruction prompts (via `isMeta: true` messages)
4. Skills **modify execution context** by changing tool permissions and model selection
5. Selection happens via **LLM reasoning**, not algorithmic matching
6. Tool permissions are **scoped to skill execution** via execution context modification
7. Skills inject two user messages per invocation—one for user-visible metadata, one for hidden instructions sent to the API

The Elegant Design: By treating specialized knowledge as *prompts that modify conversation context* and *permissions that modify execution context* rather than *code that executes*, Claude Code achieves flexibility, safety, and composability that would be difficult with traditional function calling.

References

- Introducing Agent Skills
- Equipping Agents for the Real World with Agent Skills
- Claude Code Documentation
- Anthropic API Reference
- Official Documented Frontmatter Fields
- Internal Comms Skill

- Skill Creator Skill
- ChatGPT 5 System Prompt (leaked, not official)

```
@article{
    leehanchung_bullshit_jobs,
    author = {Lee, Hanchung},
    title = {Claude Agent Skills: A First Principles Deep Dive},
    year = {2025},
    month = {10},
    day = {26},
    howpublished = {\url{https://leehanchung.github.io}},
    url = {https://leehanchung.github.io/blogs/2025/10/26/claude
}
```

Han, Not Solo

Han Lee
lee [dot] hanchung [at] gmail [dot] com

Github
Linkedin
Twitter
RSS

Han Lee's blog on machine learning engineering, compound AI systems, search and information retrieval, and recsys — exploring machine learning, LLM agents, and data science insights from startups to enterprises.